



Artificial Intelligence Fundamentals

LOCAL SEARCH

SEARCH IN NONDETERMINISTIC,
UNOBSERVABLE ENVIRONMENTS

Local Search

Enough is as good as a feast

aka «chi s'accontenta gode»

We want to be **fast**

We want to be **cheap**

We want «**good enough**» solutions (remember weighted A*?)

We are not scared about **infinite** search spaces (e.g., continuous ones)

No systematic search!

- we don't care how we get to the goal, we just want to go there.

Local search returns a solution, not the path to reach it

- It does not store visited states
- It does not explore the whole state space

Local search idea

1

Keep only one
candidate
solution (the
current one)

2

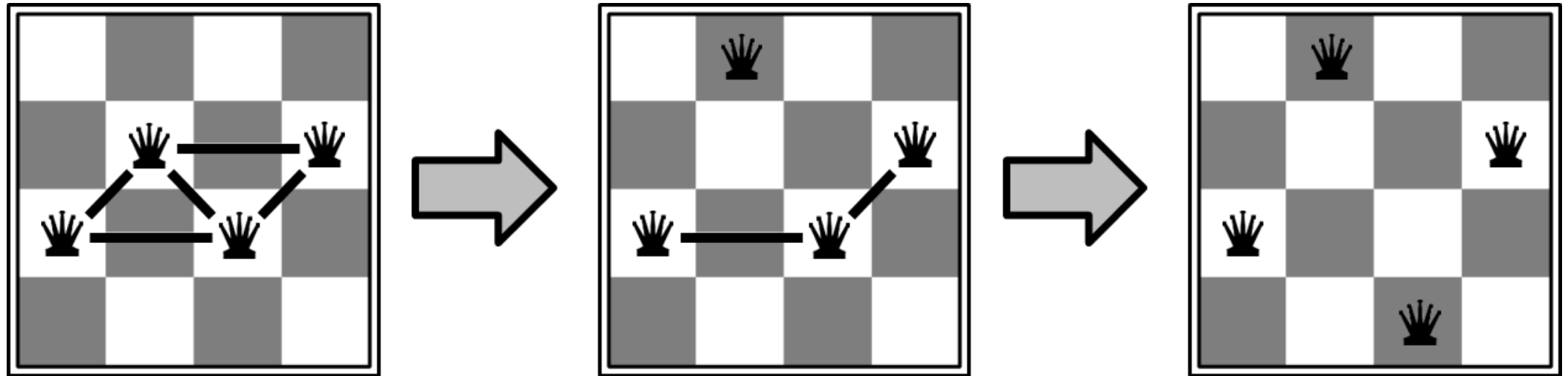
Iteratively update
it by moving in
the direction of a
better solution

3

Stop when
reaching the goal
(or close enough)

N-queens problem

Avoid queen attacking each other («collision») → move a queen to reduce number of collisions



Incredibly efficient, even for $N=1$ million

The state space landscape

Hill-climbing can get stuck

- Local optima, flat valleys

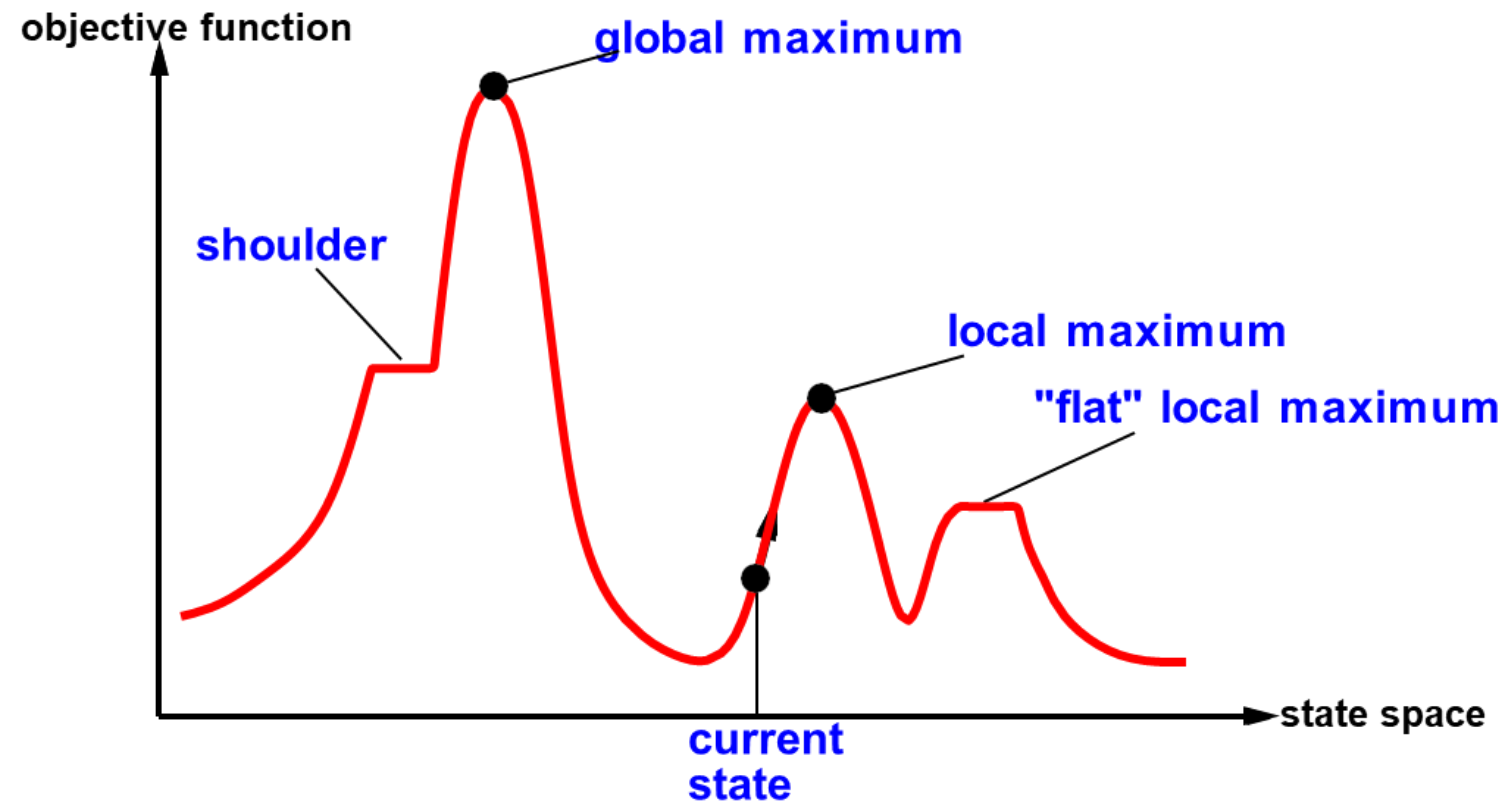
Random restart

- escape local optima
- Complete, but boy you have to wait

Sideways moves

- Escape flat valleys **if they are not a maxima**
- Fix: move sideways for max K steps

Maximize f = minimize $-f$
(you can use heuristics as objective function)



Hill-climbing (maximize value)

Greedy local search: keep only 1 candidate solution in memory

If you use the gradient of the evaluation function to guide the search → gradient descent!
The foundation of modern deep learning optimization

```
function Hill-Climbing(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← Make-Node(Initial-State [problem])
  loop do
    neighbor ← a highest-valued successor of current
    if Value[neighbor] ≤ Value[current] then return State[current]
    current ← neighbor
  end
```

Simulated annealing

Randomness is a powerful tool. Ask Prof. Gallicchio.

Idea: escape local optima by making random moves

- random walks never get stuck, but they are slow

Simulated annealing = hill climbing + random moves

- Random moves become less frequent over time
- **Temperature** parameter controls trade-off between hill-climbing and random walk
- Temperature starts high and decreases towards 0

Simulated annealing is such an algorithm. In metallurgy, **annealing** is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low-energy crystalline state.

Simulated annealing (minimize cost)

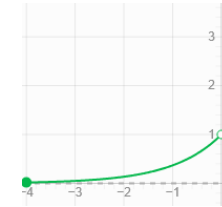
```
function Simulated-Annealing(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a function from time to “temperature”
  local variables: current, a node
                   next, a node
                   T, a “temperature” controlling prob. of downward steps

  current ← Make-Node(Initial-State [problem])
  for t ← 1 to ∞ do
    T ← schedule(t)
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{Value}[\textit{current}] - \text{Value}[\textit{next}]$ 
    if  $\Delta E > 0$  then current ← next # we picked a good solution, let's keep it
    # we picked a bad solution, let's accept it with a certain probability
    else current ← next with probability  $e^{\Delta E/T}$ 
```

Simulated annealing dynamics

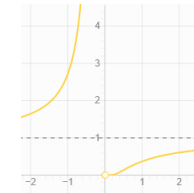
Fix T (e.g. 1), $P(\text{accepting bad move}) = e^{\Delta E}$

- Large (negative) ΔE corresponds to a very bad move $\rightarrow$ low probability



Fix ΔE (e.g. -1) $P(\text{accepting bad move}) = e^{\frac{-1}{T}}$

- Large temperature increases the probability
- $T = 0 \rightarrow$ deterministic, greedy search



Annealing schedule: start with large T and decrease it over time

- Exponential decrease: $T_t = k T_{t-1}$, $0 < k < 1$
- Power-law decrease: $T_t = \frac{T_0}{t^a}$, e.g., $a = 1$

Local Beam Search

Instead of keeping only one solution, keep K (beam size).

Expand all and select top- K states.

States interact with each other, abandoning low-value paths to focus on most promising ones.

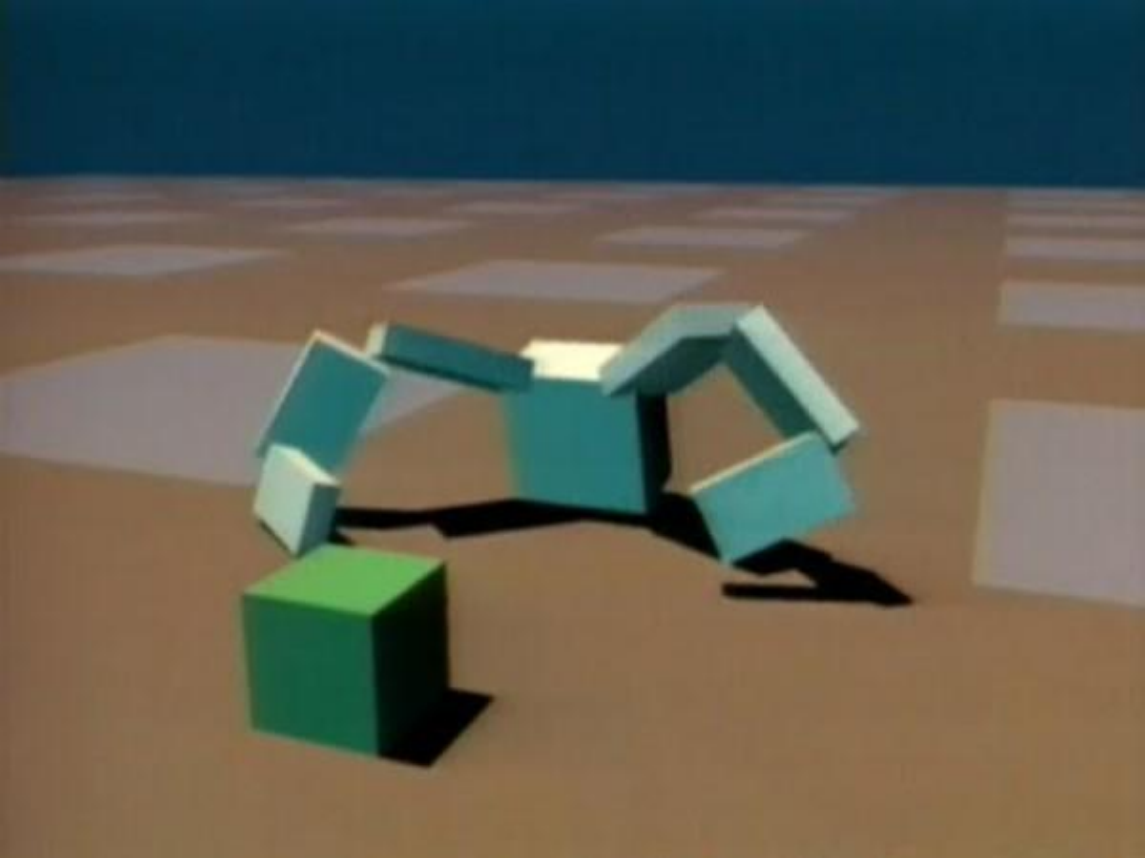
- They are not K parallel searches.

Can exhibit low diversity across states. Solutions cluster on the same local optima.

- → **stochastic beam search**: choose next states with probability proportional to their value

Evolutionary optimization

We will discuss this topic in separate lecture(s)!





e(s)!

Beam search decoder in NLP

NLP?

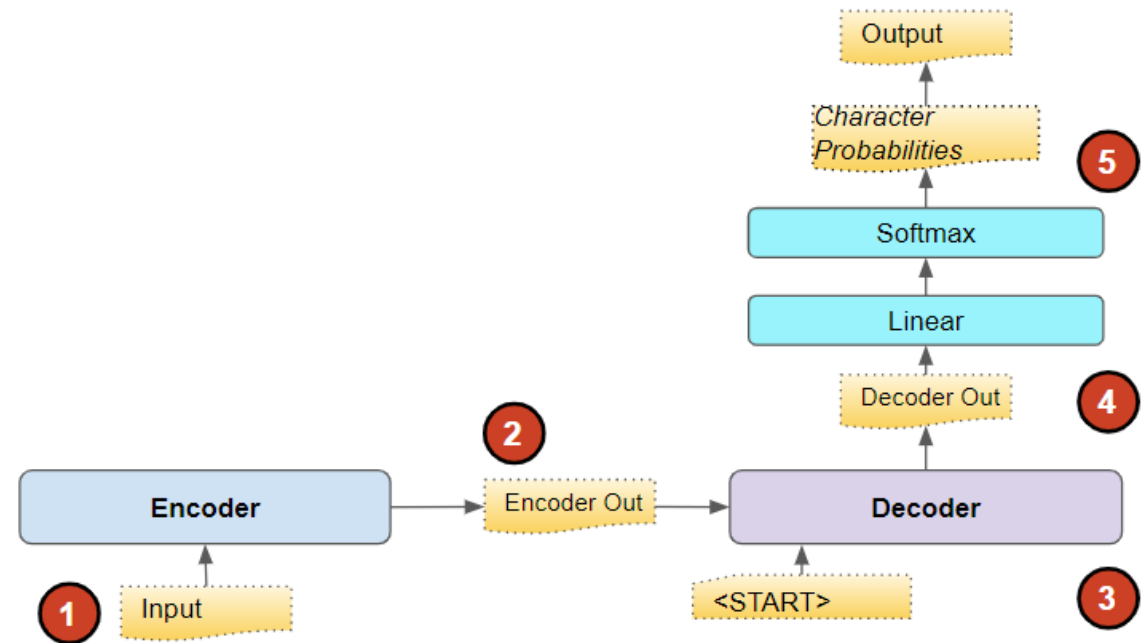
- You have a course for this...
- but I can tell you something

Let's consider the case of machine translation or, more in general, text generation

Input: how are you?

Output: come stai?

Run the decoder once for each output word

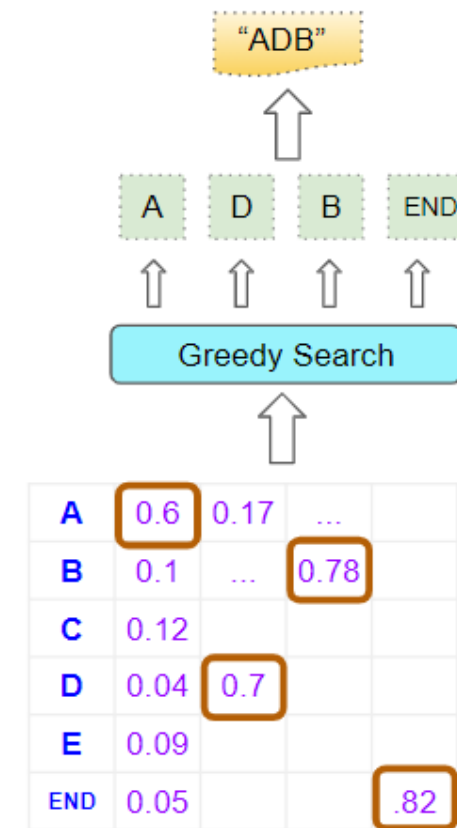


Which word to choose next? Greedy

Greedy search: just take the most likely word for each position

- Select the word with max probability
- Repeat until you select END

N output positions = N decoder runs



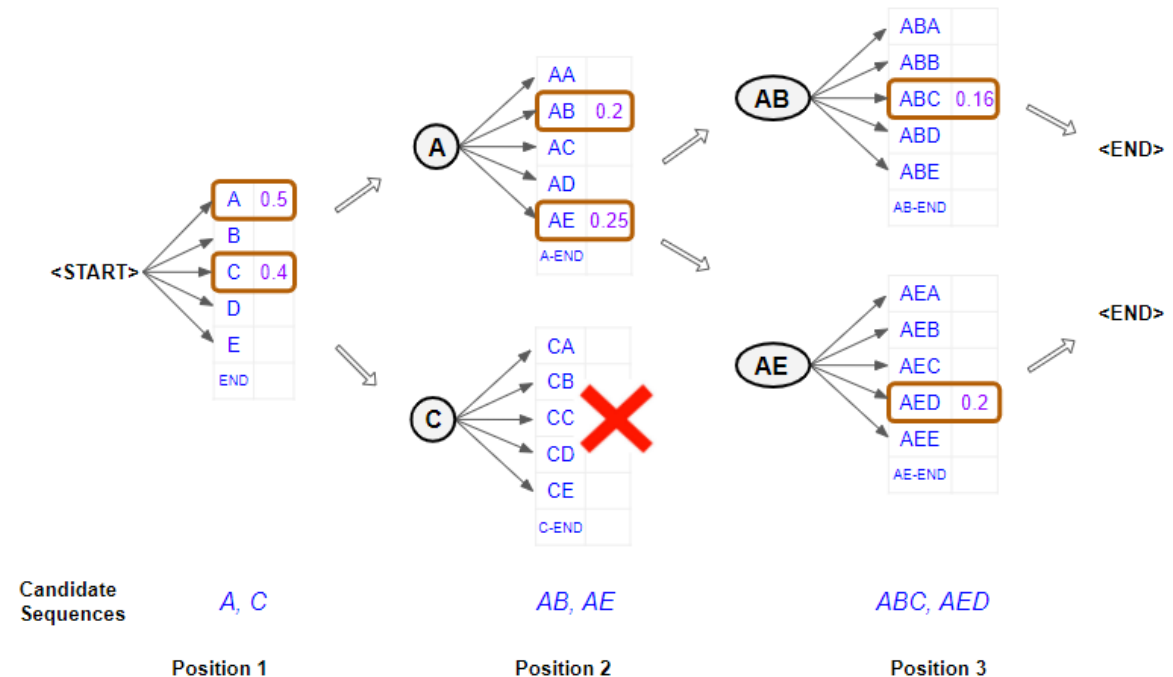
Which word to choose next? Beam

Beam search:

- Keep a beam size of K best candidates for each position
- Expand the current K best candidates and keep the next best K
- Continue until END is selected in K candidates
- Return the best out of the final K candidates

N output positions = KN decoder runs

How to select K best at each turn?
The K largest **combined** probability values!



Computing joint output probabilities

First position is easy → the probability is given by the decoder itself

- $P(A) = 0.5, P(C) = 0.4, \dots$

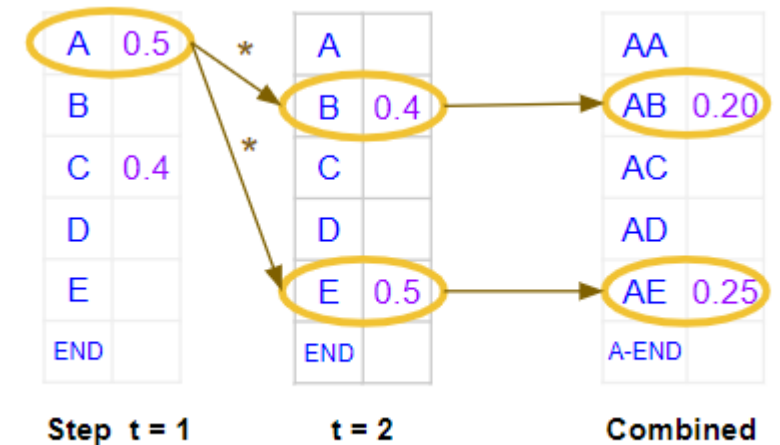
Second position is the joint probability of **both** positions

- $P(AB) = P(A)P(B|A) = 0.5 * 0.4 = 0.2$

Third position is the joint probability of all 3 positions

- $P(ABC) = P(AB)P(C|AB) = 0.2 * 0.8 = 0.16$

You compute all probabilities and keep the largest K each time



Search in Nondeterministic Environments

Nondeterministic environments

Some action's result can be non-deterministic

Belief state: a set of possible future states for each current state and action pair

- You don't know in advance where the action will take you

Conditional plan: solution is a tree (if-then-else chains), not a sequence.

- You have to run the solution and depending on the environment's result, pick the correct action
- Q: Is this open loop or closed loop?

How to search for the conditional plan: **AND-OR search trees**

- **AND** node: provided by the environment, the set of all possible next states
- **OR** node: provided by the agent, which action to execute

Erratic vacuum world

Non-deterministic action:

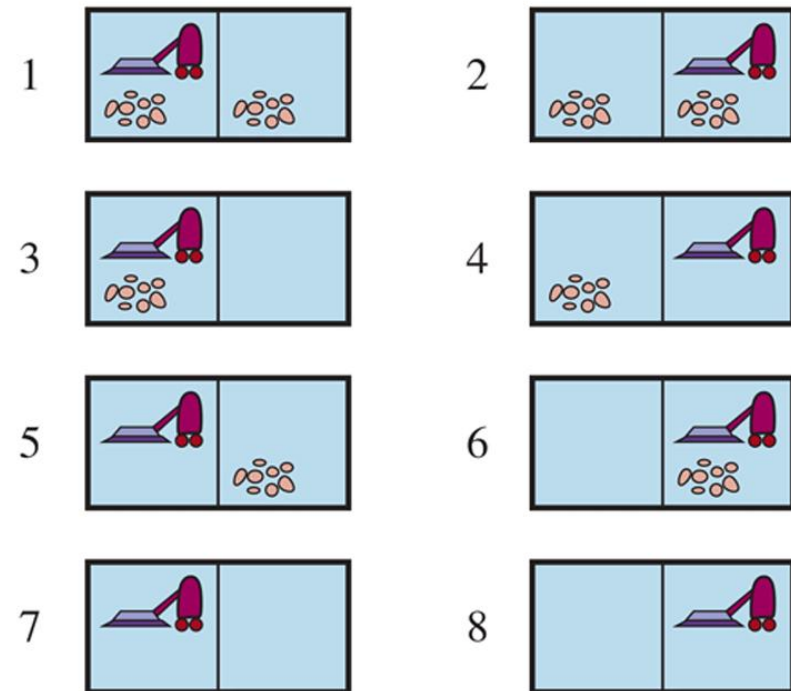
- If current state is dirty, «suck» cleans the current square, and sometimes also the adjacent square
- If current state is clean, «suck» can deposit dirt

Result(s,a)= **belief state**

- Result(1, «suck») = {5, 7}

Conditional plan: «suck»; if 5 [«right», «suck»]

- Solves the problem when starting from 1



AND-OR search tree

Search for conditional plans

- Versions of A* for AND-OR trees do exist
- We will not cover it

A **solution** is a **subtree** where

- Each leaf is a goal
- Each OR node selects one action
- Each AND node's outcomes is included

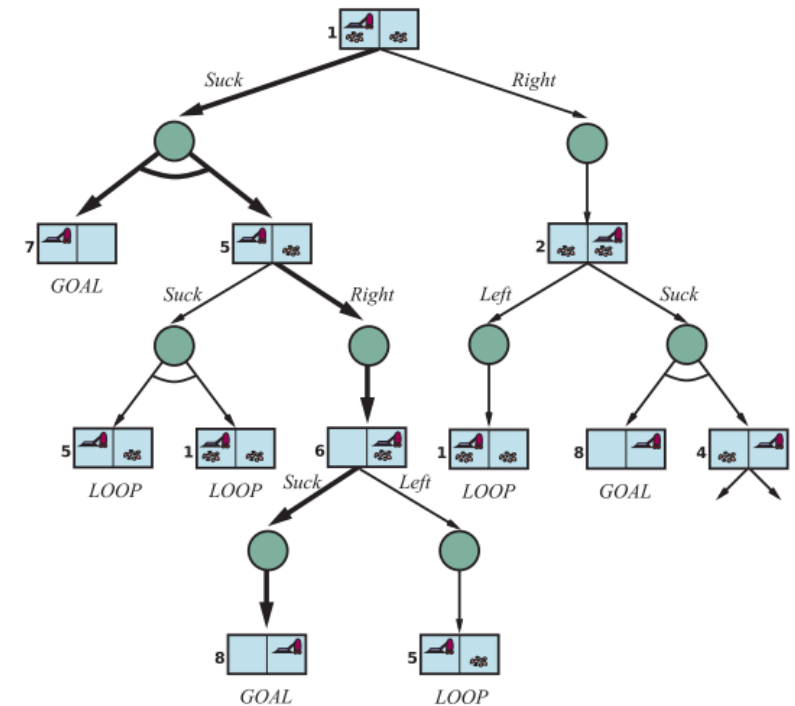


Figure 4.10 The first two levels of the search tree for the erratic vacuum world. State nodes are OR nodes where some action must be chosen. At the AND nodes, shown as circles, every outcome must be handled, as indicated by the arc linking the outgoing branches. The solution found is shown in bold lines.

AND-OR DFS search

Complete? Yes, if acyclic solutions exist

Time? $O((bk)^d)$

- k=branching factor of AND nodes
- b=branching factor of OR nodes

Space? Same as time.

Need to store the entire plan, which may contain all states at the last depth.

Optimal? No (DFS-like)

function AND-OR-SEARCH(*problem*) **returns** a conditional plan, or *failure*
return OR-SEARCH(*problem*, *problem*.INITIAL, [])

function OR-SEARCH(*problem*, *state*, *path*) **returns** a conditional plan, or *failure*
if *problem*.IS-GOAL(*state*) **then return** the empty plan
if IS-CYCLE(*path*) **then return** *failure*
for each *action* **in** *problem*.ACTIONS(*state*) **do**
 plan ← AND-SEARCH(*problem*, RESULTS(*state*, *action*), [*state*] + *path*)
 if *plan* ≠ *failure* **then return** [*action*] + *plan*
return *failure*

function AND-SEARCH(*problem*, *states*, *path*) **returns** a conditional plan, or *failure*
for each *s_i* **in** *states* **do**
 plan_i ← OR-SEARCH(*problem*, *s_i*, *path*)
 if *plan_i* = *failure* **then return** *failure*
return [**if** *s₁* **then** *plan₁* **else if** *s₂* **then** *plan₂* **else** ... **if** *s_{n-1}* **then** *plan_{n-1}* **else** *plan_n*]

Cyclic solutions

Might be found by AND-OR DFS if you don't check for loops

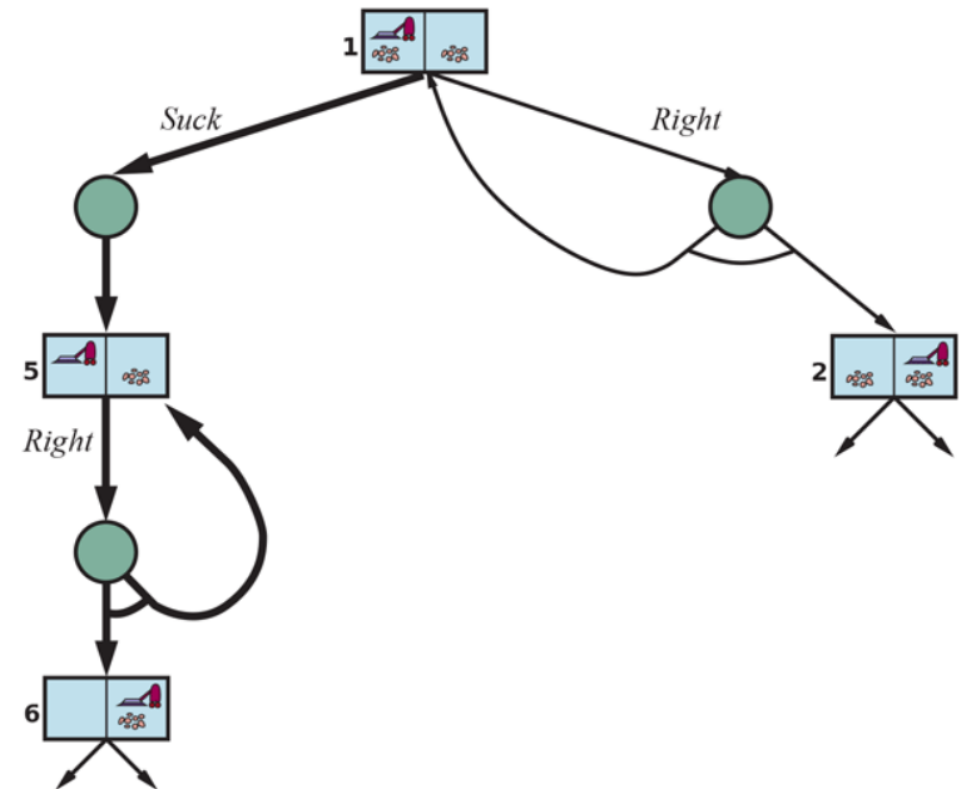
- But you lose completeness

Suppose only Left/Right actions sometimes fail

- $\text{Result}(1, \text{«right»}) = \{1, 2\}$

Cyclic solution:

[«suck», *while* 5: «right», «suck»]



Search in Unobservable Environments

Searching with no observations

Sensorless / conformant problem: no percepts

- State space is available

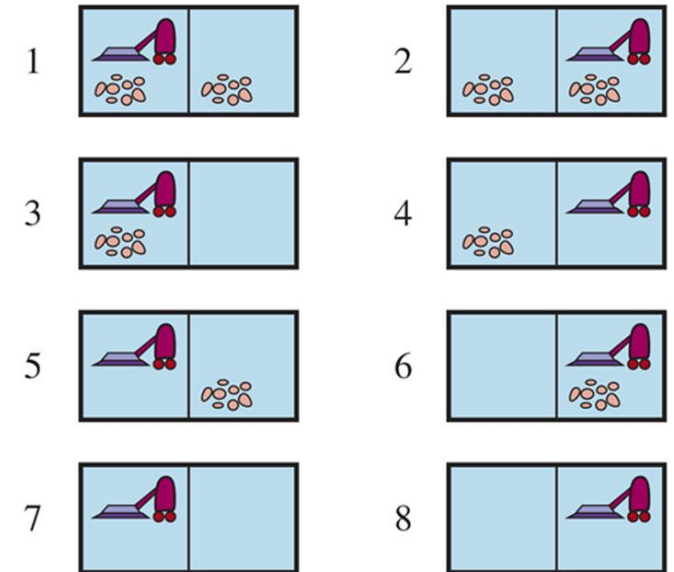
Agent's percepts need to **estimate** state (still a belief state)

Solution is a sequence of actions

- Cannot be a conditional plan as you don't get the feedback from the environment

Actions can still provide information about the possible future states

- Initial belief state = $\{1,2,3,4,5,6,7,8\}$, action=«right», next belief state = ?
- $\{2, 4, 6, 8\}$



Search in belief space

Use the search algorithms we already know, but re-define the search problem

Search in the belief space (fully-observable) rather than in the state space (unobservable)

Belief state space = powerset of states (2^N possible beliefs b)

- Initial belief state = $\{1, \dots, N\}$

$Actions(b) = \bigcup_{S \in b} Actions(S)$

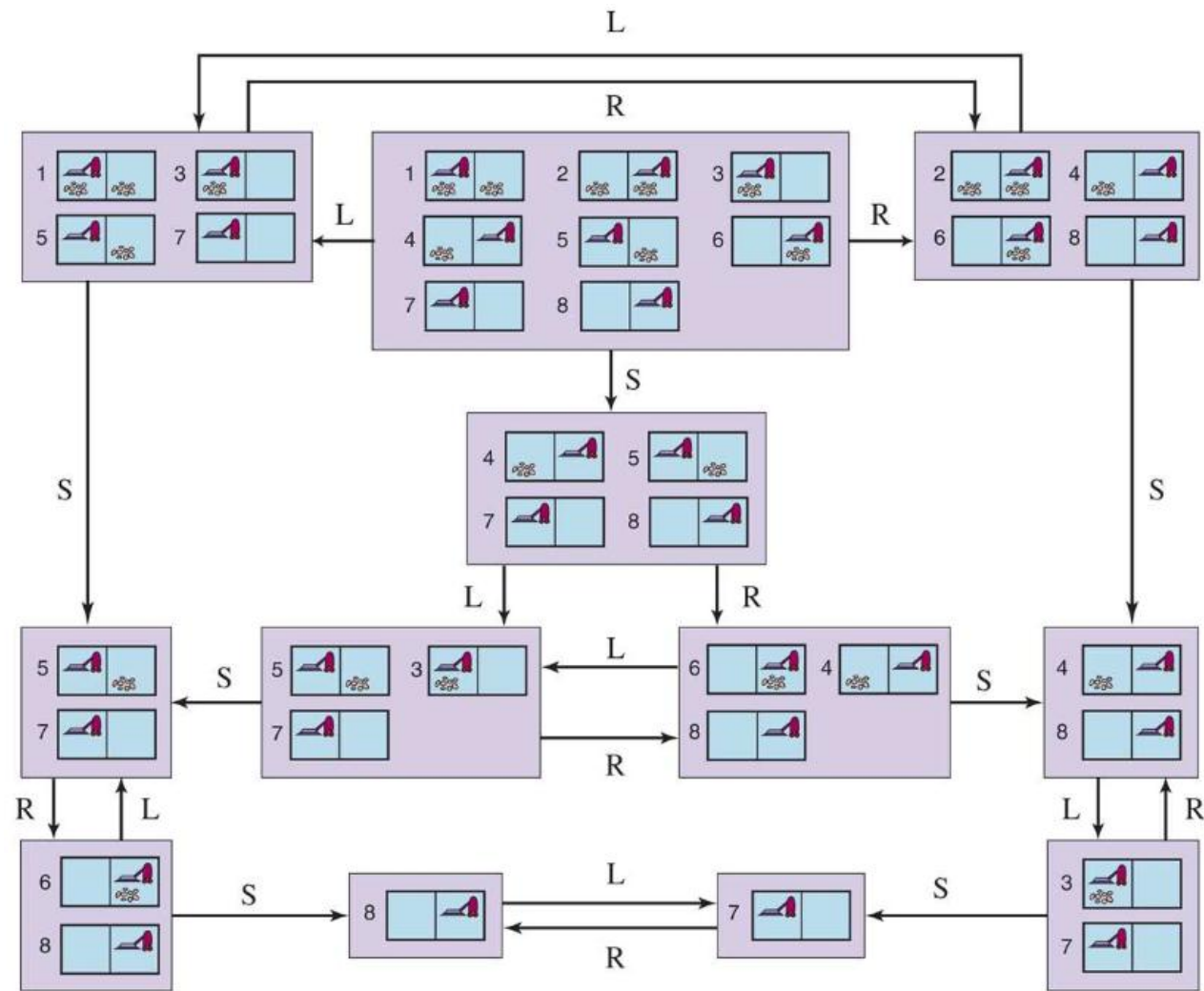
- If illegal actions are dangerous $\rightarrow$ intersection: only consider actions available in all states of the belief

$b' = Result(b, a) = \{S' \mid S' = Result(S, a), \forall S \in b\}$

- the set of all possible states resulting from the application of a to all states in the belief

Possibly/Necessarily achieves the goal if **any/all** states in the belief are a goal

Action cost: assume same cost given the same (S, a) . Otherwise, take the mean action cost (lots of alternatives here).



Making the search more efficient

$$B_1 = \{5, 7\}, \quad B_2 = \{1, 3, 5, 7\}$$

1. If you already found a solution for B_2 , prune $B_1 \rightarrow$ the solution for B_2 works in all states in B_2 , therefore also works in B_1
2. If you already found a solution for B_1 , prune $B_2 \rightarrow$ Just solve the problem by following the B_1 path instead of following B_2

Still, search in conformant problems is very expensive due to the size of the belief state space

Many problems cannot be solved without observations (8-puzzle?)

Partially-observable environments

Modeled with AND-OR search → **belief state**

Percepts are a subset of the observations: e.g., [L, Dirty].

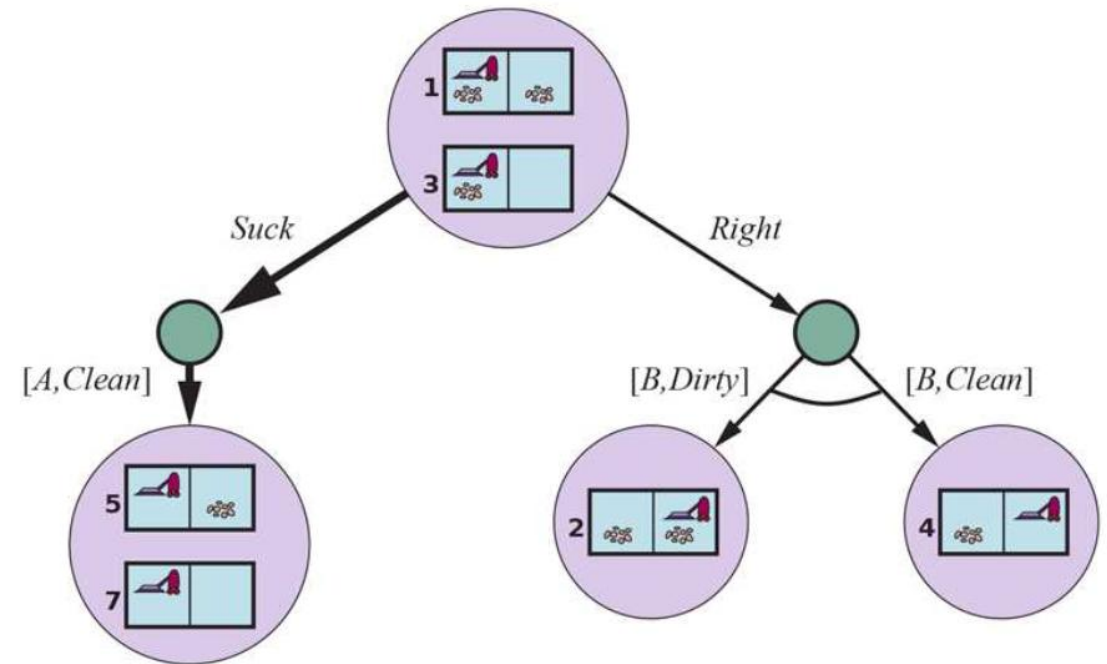
- The robot only perceives the current location

[L, Dirty] → {1, 3}

Possible percepts (instead of nondeterministic actions) give rise to **belief** states.
Same AND-OR search.

Partially-observable problems can be deterministic or stochastic as well

- As it often happens, you need to combine different ideas together



PROBLEM

Values

Upcoming – A famous
case of search

constraint